

UFMG DCC - 2026.1- Redes de Computadores

Trabalho Prático 1 - Jogo TERMO

Discente: Lucas Bernardes Pena (2026089544)

Curso: Ciência de Dados

1. Introdução

Neste trabalho prático foi codificado um programa simples em C para o jogo TERMO, que consiste em adivinhar uma palavra secreta de 5 letras definida na inicialização do servidor. O jogador interage pelo programa cliente, que se conecta a um IP e uma porta conhecidos e envia palpites ao servidor. A cada palpite, o servidor responde com uma mensagem de estrutura padronizada (definida no protocolo do enunciado), contendo dicas sobre as letras que estão na posição correta, as que existem na palavra mas em outra posição e as que não fazem parte dela. O jogo termina quando o cliente acerta a palavra completa.

A comunicação é feita via TCP, utilizando a interface POSIX de sockets, e o sistema suporta tanto IPv4 quanto IPv6. O projeto é composto por três arquivos: `server.c`, `client.c` e `protocolo.h`. O arquivo de contrato contém a estrutura `GameMessage`, o enumerador `MessageType` e as funções auxiliares de envio e recebimento compartilhadas pelos dois programas. A compilação é feita pelo `makefile` com o comando `make`, que gera os binários `client` e `server` no `root` do projeto.

2. Implementação

2.1 Servidor

O servidor recebe três argumentos: o protocolo ("`v4`" ou "`v6`"), a porta e a palavra secreta. A palavra é validada (exatamente 5 letras de A a Z) e convertida para maiúsculas. Em seguida, o endereço é configurado de acordo com o protocolo escolhido e o servidor executa a sequência `socket`, `bind`, `listen` e `accept`, aguardando a conexão de um cliente. Após a conexão, o servidor envia uma mensagem do tipo `MSG_START` e entra em um laço no qual recebe os palpites (`MSG_GUESS`):

- `MSG_ERROR`: palpite inválido (não conta tentativa);
- `MSG_FEEDBACK`: Palpite válido, incrementa o contador de tentativas e calcula o vetor `feedback[5]` (2 para posição correta, 1 para letra presente em outra posição e 0 para letra ausente);
- `MSG_WIN`: acerto e encerra o jogo;

Ao final, o servidor fecha os sockets e imprime "`Cliente desconectado.`".

2.2 Cliente

O cliente recebe o endereço IP e a porta do servidor. O tipo do endereço (IPv4 ou IPv6) é descoberto automaticamente, o socket é criado com a família correspondente e a conexão é feita com `connect`. Após receber o `MSG_START`, o cliente lê o input, converte as letras minúsculas para maiúsculas, envia `MSG_GUESS` e exibe a resposta do servidor no formato exigido pelo enunciado.

3. Dificuldades e soluções adotadas

3.1 Configuração dos sockets para cada tipo de IP

A primeira dificuldade foi configurar os sockets para funcionar tanto com IPv4 quanto com IPv6, já que cada versão usa uma estrutura de endereço diferente (`sockaddr_in` e `sockaddr_in6`), com campos de nomes distintos (`sin_family` ou `sin6_family`, `sin_addr` ou `sin6_addr`) e tamanhos diferentes. Para evitar duplicar todo o código de criação do socket, foi utilizada a estrutura genérica `sockaddr_storage`, que tem espaço suficiente para qualquer um dos dois tipos. Dependendo do protocolo, ela é tratada como `sockaddr_in` ou `sockaddr_in6` para preencher família, porta e endereço, e o tamanho correto é guardado em uma variável `addrlen`. A partir daí, as chamadas `socket`, `bind` e `connect` são as mesmas para os dois casos, usando `addr.ss_family` como família do socket.

3.2 Parsing do IP para definir o protocolo no cliente

No servidor o protocolo é informado diretamente como argumento, mas o cliente recebe apenas o endereço IP, sendo necessário descobrir a partir dele qual versão do protocolo usar. A solução foi utilizar a função `inet_pton`, que converte o texto do endereço para o formato binário e retorna 1 em caso de sucesso. O cliente tenta primeiro interpretar o endereço como IPv4 (`AF_INET`); se falhar, tenta como IPv6 (`AF_INET6`); se ambos falharem, o endereço é considerado inválido.

3.3 Uso de caracteres como inteiros na passagem de dados

O protocolo define o palpite como um vetor de inteiros (`int guess[5]`) e o feedback também como inteiros (`int feedback[5]`), enquanto o usuário digita uma string. Foi necessário compreender que, em C, um caractere é apenas um número (seu código ASCII), de forma que cada letra digitada pode ser copiada diretamente para uma posição do vetor `guess`, e as comparações entre o palpite e a palavra secreta (que é uma string) funcionam normalmente. Isso também permitiu fazer a conversão de minúsculas para maiúsculas sem bibliotecas adicionais,

apenas com aritmética: se o caractere está entre 'a' e 'z', basta somar 'A' - 'a'. A validação segue a mesma ideia, verificando se cada valor está entre 'A' e 'Z'.

Uma primeira versão da função de verificação do palpite tentava montar a resposta como uma string de caracteres (letra, '*' ou '_') e retornar ao mesmo tempo o número de acertos e essa string, o que não é possível em C (uma função não retorna dois valores). A solução foi seguir o protocolo: a função recebe o vetor **feedback** como parâmetro, preenche-o com os códigos 2, 1 e 0 e retorna apenas o número de acertos exatos. A conversão dos códigos para os símbolos exibidos ficou a cargo do cliente.

3.4 Compreensão das funções de socket

Foi necessário entender o papel de cada função da interface de sockets e a ordem em que devem ser chamadas:

- **socket**: cria o descritor do socket, recebendo a família (**AF_INET** ou **AF_INET6**) e o tipo (**SOCK_STREAM**, correspondente ao TCP);
- **bind**: associa o socket a um endereço e porta locais, no servidor;
- **listen**: coloca o socket em modo de escuta, aguardando conexões;
- **accept**: bloqueia até que um cliente se conecte e **retorna um novo descritor**, usado para a comunicação com esse cliente, enquanto o socket original continua apenas escutando;
- **connect**: usada no cliente para estabelecer a conexão com o servidor.

Uma dúvida inicial foi justamente a diferença entre o socket de escuta e o socket retornado pelo **accept**: as mensagens do jogo são enviadas e recebidas pelo descritor do cliente, e ao final os dois precisam ser fechados com **close**. Durante os testes também foi observado que, ao reiniciar o servidor logo após encerrar uma partida, o **bind** pode falhar com "**Address already in use**", pois o sistema operacional mantém a porta reservada por um curto período após o fechamento da conexão TCP. Nesses casos, basta aguardar alguns instantes ou usar outra porta.

3.5 Uso do **recv** e **send**

Inicialmente não estava claro como usar **recv** e **send** para trocar a estrutura **GameMessage**. Essas funções recebem um ponteiro para um buffer e um tamanho em bytes, então é possível enviar a estrutura inteira passando seu endereço (**&msg**) e **sizeof(GameMessage)**. A dificuldade é que o TCP é um fluxo contínuo de bytes, e uma única chamada de **recv** pode retornar menos bytes do que o solicitado. Para garantir que a mensagem chegue completa, foram criadas as

funções `recv_all` e `send_all`, que repetem a chamada em um laço até transferir todos os bytes. A `recv_all` também retorna 0 quando a outra ponta fecha a conexão, o que o servidor usa para sair do laço do jogo. Essas funções foram inicialmente escritas apenas no servidor e chegaram a ficar duplicadas no código, causando erro de compilação, além de não estarem disponíveis no cliente. A solução foi movê-las para o arquivo `protocolo.h`, incluído pelos dois programas.

4. Conclusão

O trabalho permitiu compreender na prática o funcionamento da comunicação cliente-servidor com sockets TCP em C, desde a configuração de endereços IPv4 e IPv6 até a troca de mensagens estruturadas com `send` e `recv`. As principais dificuldades estiveram na configuração dos endereços para as duas versões do protocolo IP, na representação de caracteres como inteiros exigida pelo protocolo, na regra de contagem de letras repetidas e na garantia de que as mensagens fossem transmitidas por completo. As soluções adotadas buscaram manter o código simples e fiel ao que é pedido no enunciado, garantindo a interoperabilidade com implementações de outros alunos.